Genrikh Vetsler

Machine Learning Project 2

Predicting Customer Churn with Machine Learning.

Contents

1 Introduction **2**

2 Data **3**

2.1 Target distribution and class imbalance 3

2.2 Encoding, feature selection, and multicollinearity 4

2.3 Summary statistics 4

2.4 Scaling and normalized feature space 5

2.5 Low-dimensional visualization (PCA) 6

3 Methodology **7**

3.1 Models 7

4 Results **9**

4.1 Linear models 9

4.1.1 Logistic Regression 9

4.1.2 Ridge Classifier 10

4.2 Tree-based models 12

4.2.1 Decision Tree Classifier 12

4.2.2 Random Forest Classifier 13

4.2.3 Bagging Classifier 15

4.3 Support Vector Machine 16

4.4 K-Nearest Neighbors 17
4.5 Combined results 18

5 Future work **19**

6 Conclusion **20**

1 Introduction

Customer churn is a central business problem in subscription industries, and it is especially important in telecommunications, where switching costs can be low and competition is intense. From a firm perspective, losing an existing customer implies both the loss of future recurring revenue and additional acquisition costs to replace that customer. As a result, accurate churn prediction is a key input for retention-oriented decision-making in product management and marketing analytics, such as targeted incentives and customer success interventions.

In this project, I develop and compare supervised machine learning models to predict churn using the publicly available *Telecom Churn* dataset. The dataset contains customer-level information on account characteristics, service plans, usage intensity, and customer support interactions (e.g., account length, international and voice mail plans, day/evening/night minutes and calls, international usage, and customer service calls). The outcome variable is a binary churn indicator, where 1 denotes a customer who churned and 0 denotes a customer who remained.

The prediction task is to estimate the probability that a given customer will churn based on observed parameters. This is a decision-relevant classification problem because misclassification costs are asymmetric: failing to identify likely churners corresponds to missed opportunities for intervention, while incorrectly flagging non-churners can waste retention resources. Therefore, model evaluation goes beyond overall accuracy and focuses on discrimination performance using the ROC curve and the Area Under the Curve (AUC), which summarizes performance across classification thresholds and is informative under class imbalance.

From a learning and professional development perspective, this project benchmarks several core classification methods discussed in class, including linear models, tree-based

methods, instance-based learning, and margin-based classifiers. The analysis compares models using consistent validation procedures and interprets results in the context of churn prediction as an operational tool for customer retention.

2 Data

This project uses a customer-level telecommunications dataset containing account attributes, service subscriptions, usage measures, and a binary churn indicator. The dependent variable is `Churn`, where 1 indicates that the customer churned and 0 indicates that the customer remained. The features combine: (i) account information (e.g., account length), (ii) service plan indicators (e.g., international plan, voice mail plan), (iii) usage intensity variables (e.g., day/evening/night minutes and calls), and (iv) customer support interactions (customer service calls).

2.1 Target distribution and class imbalance

A key property of this dataset is that churn events are relatively rare. Figure 1 shows a clear class imbalance, with non-churn observations constituting the majority of the sample. In the training split, the mean of `Churn` is approximately 0.146 (Table 1), which implies that roughly 15% of customers churn in the training data. This imbalance is important for model evaluation: accuracy alone may be misleading because a classifier that predicts "no churn" for all observations can still achieve a high accuracy. For this reason, ROC curves and AUC are used later to assess discriminative performance across thresholds.

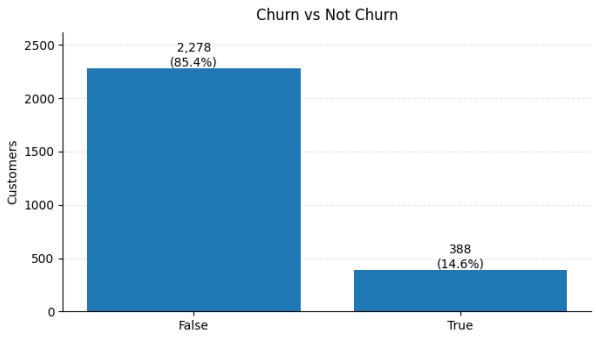


Figure 1: Column churn histogram

2.2 Encoding, feature selection, and multicollinearity

Several variables are categorical (e.g., state, area code, and plan indicators) and therefore require encoding prior to training most machine learning models. Binary categories (such as `International plan` and `Voice mail plan`) are represented as 0/1 indicators, while multi-category variables are encoded into numeric form to ensure compatibility with the model pipeline.

In addition, the dataset contains variables that are mechanically related (for example, charges are computed from minutes under a fixed rate schedule). This creates near-perfect correlations between minutes and corresponding charges (e.g., total day minutes vs. total day charge), which can introduce redundancy and instability in certain models (especially linear models). To reduce multicollinearity and avoid duplicating the same signal, the charge variables were removed and the minutes variables were retained.

Figure 2 provides an overview of the correlation structure prior to scaling. The most prominent correlations occur among usage-related measures and their closely associated counterparts.

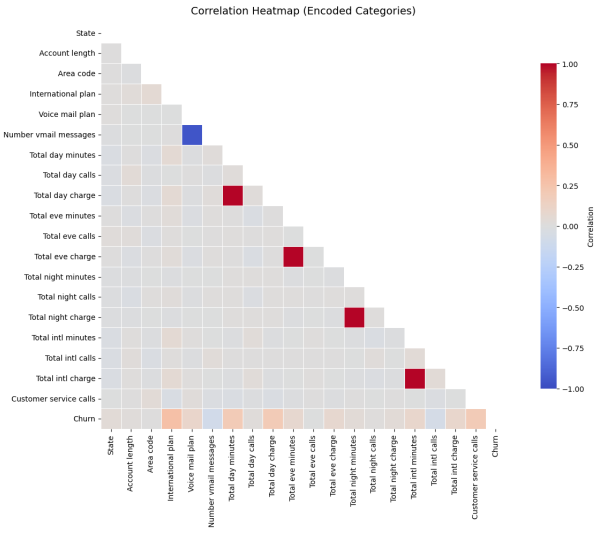


Figure 2: Correlation Heatmap

2.3 Summary statistics

Table 1 reports summary statistics for the training split. Usage variables such as day/evening/night minutes exhibit substantial dispersion, indicating meaningful heterogeneity in customer behavior. Plan indicators are binary and therefore have means that

directly correspond to prevalence rates in the training data. Customer service calls are right-skewed, with most customers making few calls and a smaller subset making many calls, which is consistent with the hypothesis that repeated support interactions may be associated with dissatisfaction and churn.

Table 1: Training Variable Summary Statistics

Feature	**Count**	**Mean**	**Std**	**Min**	**25%**	**50%**	**75%**	**Max**
Account length	2666.0	100.620	39.564	1.0	73.000	100.00	127.000	243.0
International plan	2666.0	0.101	0.302	0.0	0.000	0.00	0.000	1.0
Voice mail plan	2666.0	0.275	0.447	0.0	0.000	0.00	1.000	1.0
Number vmail messages	2666.0	8.022	13.612	0.0	0.000	0.00	19.000	50.0
Total day minutes	2666.0	179.482	54.210	0.0	143.400	179.95	215.900	350.8
Total day calls	2666.0	100.310	19.988	0.0	87.000	101.00	114.000	160.0
Total eve minutes	2666.0	200.386	50.952	0.0	165.300	200.90	235.100	363.7
Total eve calls	2666.0	100.024	20.161	0.0	87.000	100.00	114.000	170.0
Total night minutes	2666.0	201.169	50.780	43.7	166.925	201.15	236.475	395.0
Total night calls	2666.0	100.106	19.418	33.0	87.000	100.00	113.000	166.0
Total intl minutes	2666.0	10.237	2.788	0.0	8.500	10.20	12.100	20.0
Total intl calls	2666.0	4.467	2.456	0.0	3.000	4.00	6.000	20.0
Customer service calls	2666.0	1.563	1.311	0.0	1.000	1.00	2.000	9.0
Churn	2666.0	0.146	0.353	0.0	0.000	0.00	0.000	1.0

2.4 Scaling and normalized feature space

Because the numeric features operate on different scales (e.g., minutes vs. call counts vs. binary indicators), the continuous variables were standardized prior to training scale-sensitive models (e.g., Logistic Regression, SVM, and KNN). To further understand how churn relates to feature distributions, Figure 3 compares normalized feature densities by churn status. We can see that for most of the variables churn and not churn overlap.

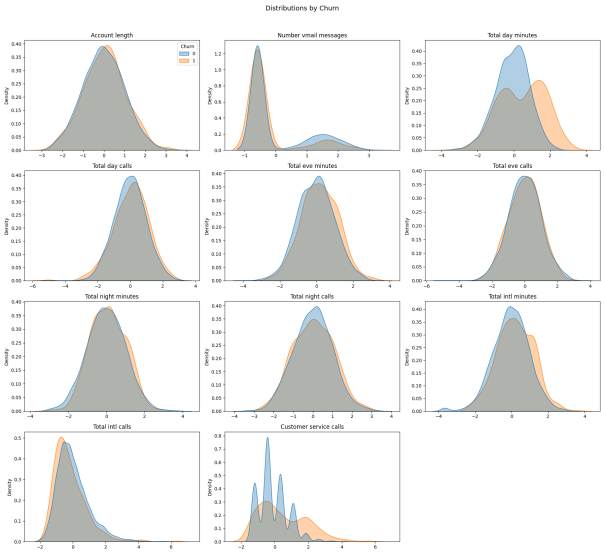


Figure 3: Distributions by churn after normalization

2.5 Low-dimensional visualization (PCA)

Finally, Principal Component Analysis (PCA) is used as an exploratory tool to visualize the dataset in two dimensions. Figure 4 plots the first two principal components and colors points by churn label. While PCA is not used as the primary predictive model, it provides an intuitive view of overlap between classes and suggests that churn and non-churn observations are not perfectly separable in a low-dimensional linear projection.

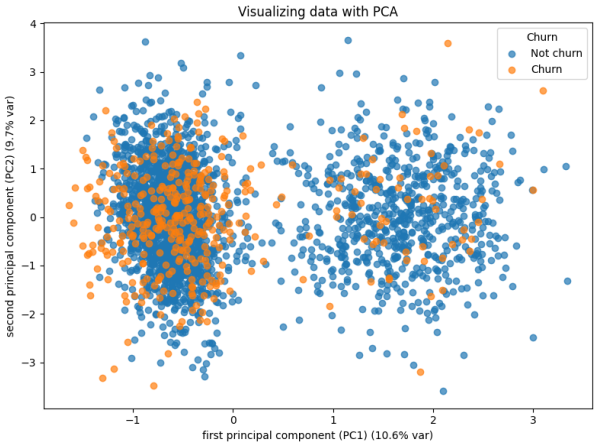


Figure 4: PCA (2 components) visualization

3 Methodology

Hyperparameters were selected via 10-fold cross-validation using ROC–AUC as their scoring metric (except Logistic Regression, which is considered as a baseline, as well as the percentage of majority class, which is (85%). Training and test performance, confusion matrix, and the ROC curve with the AUC value are reported. For tree-based models and linear models, a feature importance diagram is reported.

3.1 Models

Logistic Regression

Logistic Regression is used as a baseline probabilistic linear classifier for binary classification. It models the conditional probability of the positive class using the logistic (sigmoid) function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\beta_0 + \mathbf{x}^\top \boldsymbol{\beta}),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$. Model parameters are estimated by maximizing the regularized log-likelihood (when regularization is enabled). As a baseline, default hyper-parameters are used, and performance is reported via ROC-AUC, confusion matrix, and train/test scores.

Decision Tree Classifier

A Decision Tree Classifier is a non-parametric model that partitions the feature space into axis-aligned regions by recursively selecting splits that maximize class separation. Splits are chosen according to an impurity criterion (e.g., Gini impurity or entropy), and terminal nodes assign a class label (or class probability) based on the majority class. Key hyper-parameters tuned via 10-fold cross validation include maximum depth, minimum samples per split, and minimum samples per leaf. Feature importance is computed based on the total impurity reduction contributed by each feature across the tree.

Random Forest Classifier

Random Forest is an ensemble of decision trees trained using bootstrap samples of the training data, combined with random feature selection at each split to reduce correlation among trees. Predictions are aggregated by majority voting (and class probabilities by averaging), which typically improves generalization and reduces variance relative to a single tree. Hyper-parameters tuned via 10-fold cross validation include the number of

trees, maximum depth, and the number of features considered at each split. Feature importance is reported using mean decrease in impurity aggregated over all trees.

Support Vector Machine

Support Vector Machine is a margin-based classifier that finds a decision boundary maximizing the separation margin between classes. For non-linearly separable data, SVM employs kernel functions (e.g., RBF) to implicitly map features into a higher-dimensional space. The optimization objective balances margin maximization and classification errors through the regularization parameter C. Hyper-parameters tuned via 10-fold cross validation include C and kernel-specific parameters (e.g., γ for the RBF kernel). Model performance is evaluated using ROC-AUC, confusion matrix, and train/test scores.

In this project, linear and RBF kernels were used, best one was also tuned during the cross validation.

K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning method that classifies a sample by the majority label among its k nearest neighbors according to a chosen distance metric (commonly Euclidean distance). The method can also produce class probabilities based on neighbor class proportions. Hyper-parameters tuned via 10-fold cross validation include the number of neighbors k, the distance metric, and the weighting scheme (uniform vs. distance-weighted). Performance is reported using ROC-AUC, confusion matrix, and train/test scores.

Ridge Classifier

Ridge Classifier is a linear classification approach that uses L_2-regularized loss to prevent overfitting and handle multicollinearity. In practice, it is commonly implemented by fitting a linear model with squared loss and applying a decision threshold on the resulting scores. The regularization strength controls the bias-variance trade-off. Hyper-parameters tuned via 10-fold cross validation include the regularization parameter α. Feature importance is reported via the magnitude of the learned coefficients.

Bagging Classifier

Bagging (Bootstrap Aggregating) is an ensemble technique that trains multiple base learners on different bootstrap samples of the training data and aggregates their predictions (majority vote for classification). This approach primarily reduces variance and improves robustness, especially for high-variance base estimators (e.g., decision trees).

Hyper-parameters tuned via 10-fold cross validation include the number of estimators and the choice/configuration of the base learner. When the base learner supports it (e.g., trees), feature importance is reported based on the underlying estimator's importance measure (or via permutation importance if applicable).

4 Results

4.1 Linear models

4.1.1 Logistic Regression

Logistic Regression – Training accuracy: 86.20%

Logistic Regression – Test accuracy: 85.31%

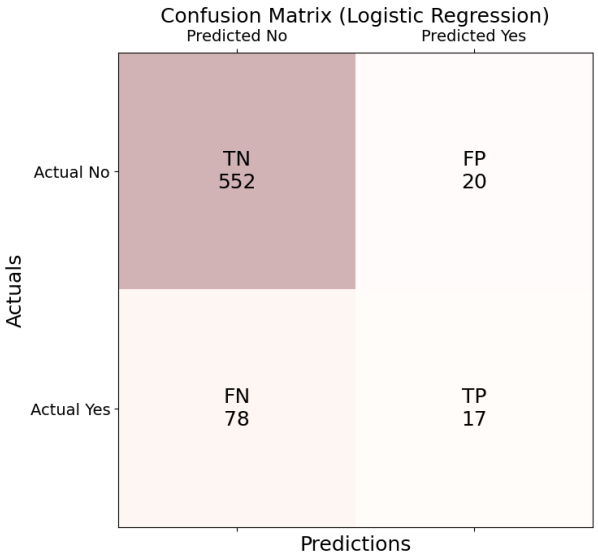


Figure 5: Logistic Regression confusion matrix (test set).

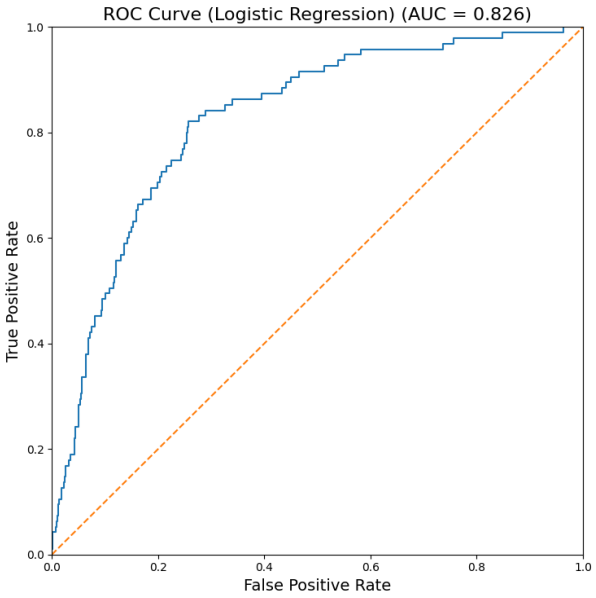


Figure 6: Logistic Regression ROC curve (test set).

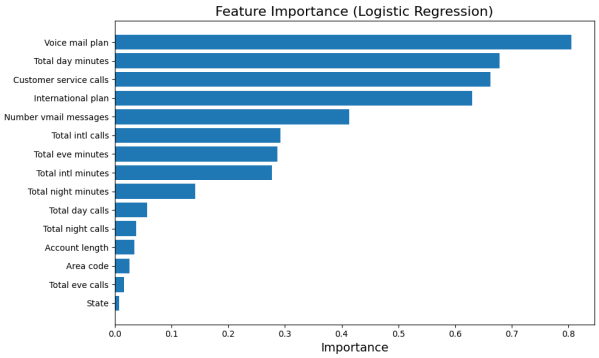


Figure 7: Logistic Regression feature importance.

4.1.2 Ridge Classifier

Ridge Classifier – Training accuracy: 86.12%

Ridge Classifier – Test accuracy: 85.61%

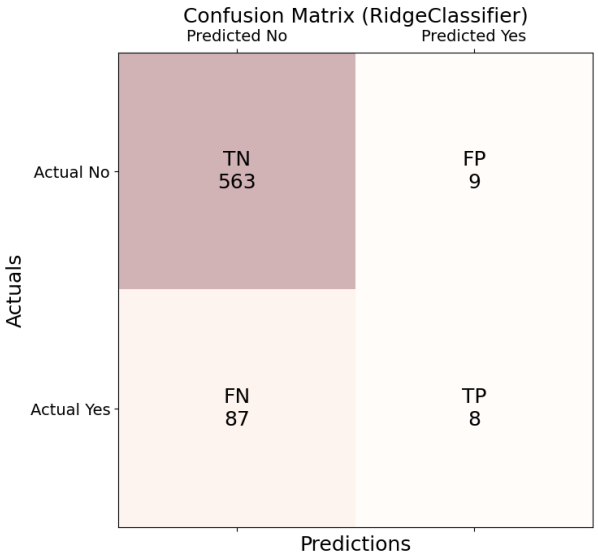


Figure 8: Ridge Classifier confusion matrix (test set).

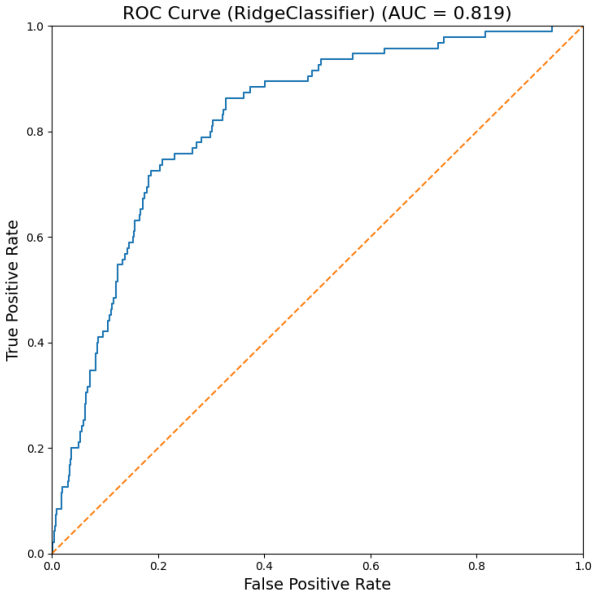


Figure 9: Ridge Classifier ROC curve (test set).

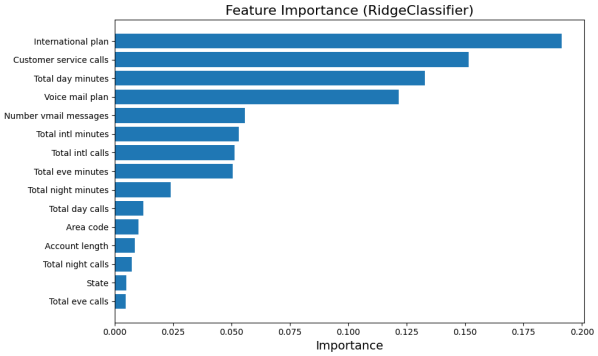


Figure 10: Ridge Classifier feature importance.

4.2 Tree-based models

4.2.1 Decision Tree Classifier

Decision Tree Classifier – Training accuracy: 97.60%

Decision Tree Classifier – Test accuracy: 93.25%

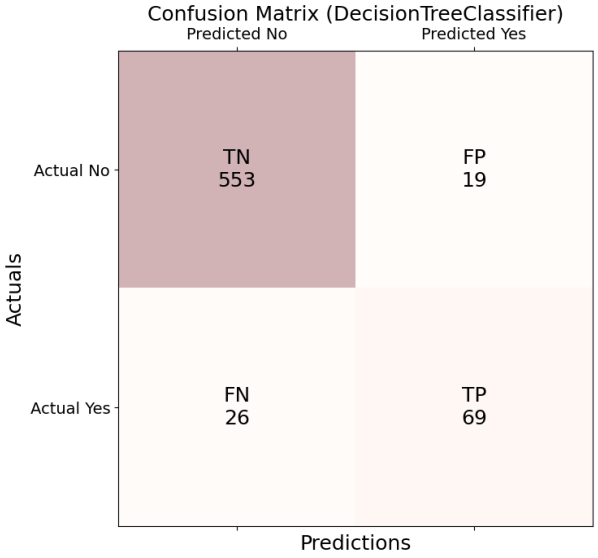


Figure 11: Decision Tree confusion matrix (test set).

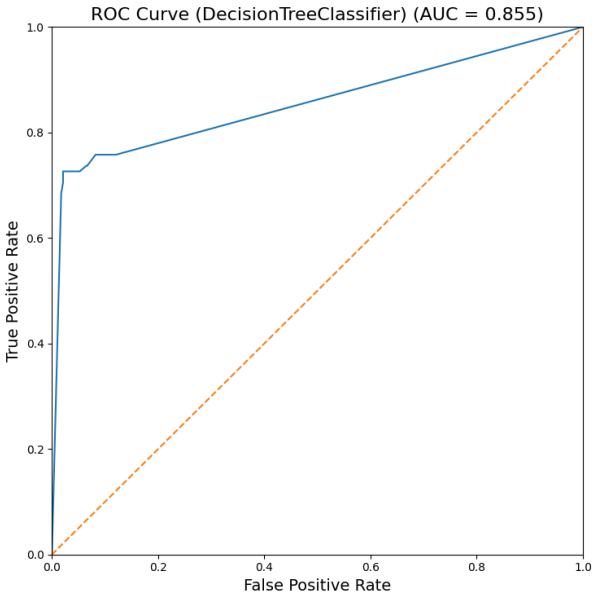


Figure 12: Decision Tree ROC curve (test set).

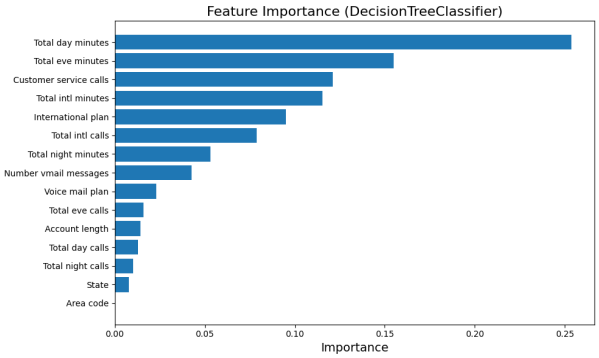


Figure 13: Decision Tree feature importance.

4.2.2 Random Forest Classifier

Random Forest Classifier – Training accuracy: 97.75%

Random Forest Classifier – Test accuracy: 93.85%

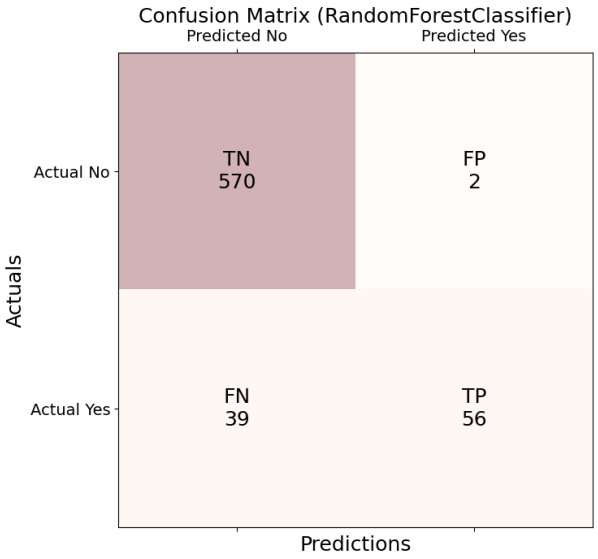


Figure 14: Random Forest confusion matrix (test set).

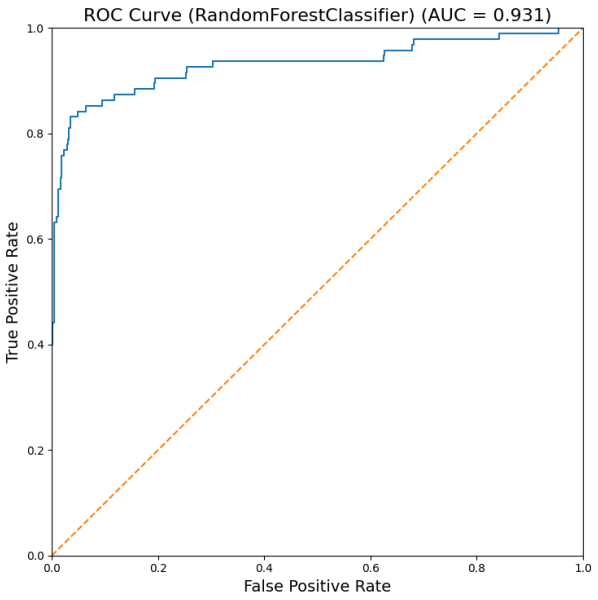


Figure 15: Random Forest ROC curve (test set).

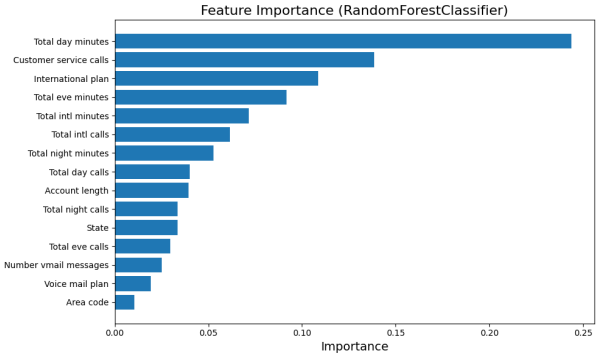


Figure 16: Random Forest feature importance.

4.2.3 Bagging Classifier

Bagging Classifier – Training accuracy: 99.96%

Bagging Classifier – Test accuracy: 95.80%

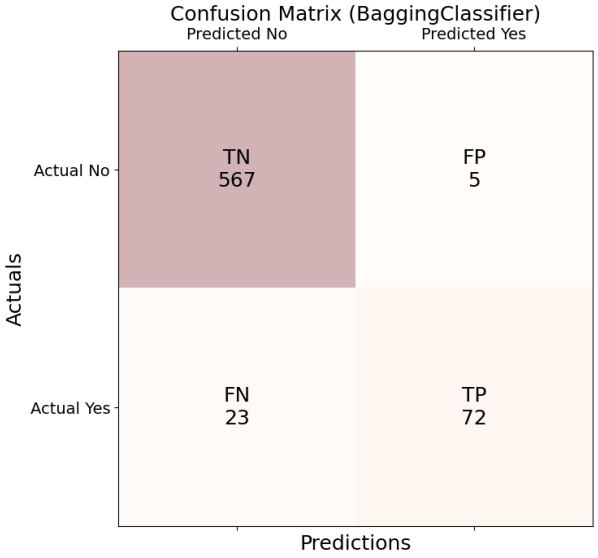


Figure 17: Bagging Classifier confusion matrix (test set).

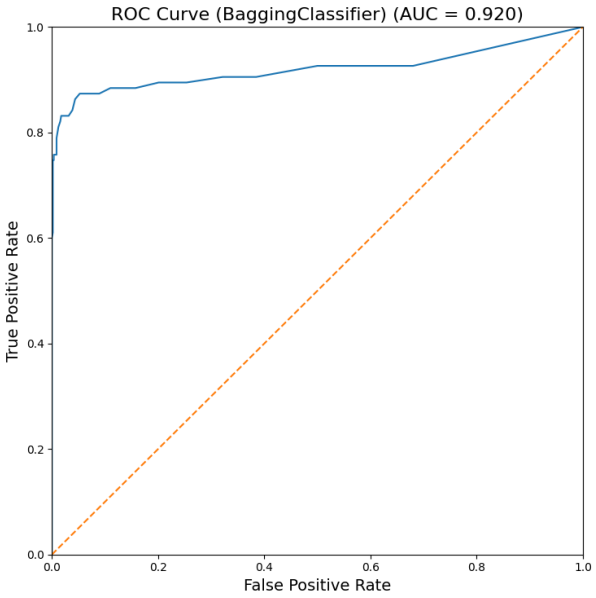


Figure 18: Bagging Classifier ROC curve (test set).

4.3 Support Vector Machine

The best kernel was RBF.

SVM – Training accuracy: 94.11%

SVM – Test accuracy: 91.60%

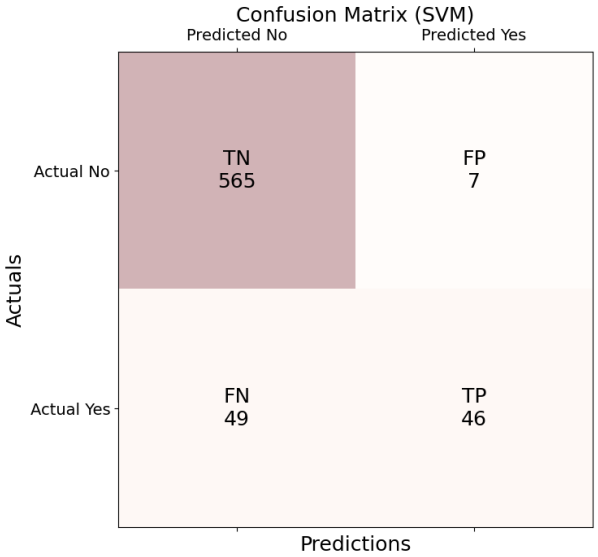


Figure 19: SVM confusion matrix (test set).

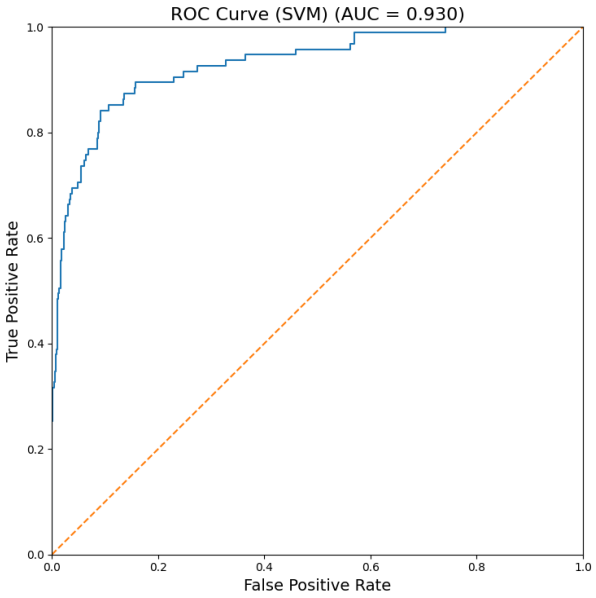


Figure 20: SVM ROC curve (test set).

4.4 K-Nearest Neighbors

K-Nearest Neighbors – Training accuracy: 88.71%

K-Nearest Neighbors – Test accuracy: 87.26%

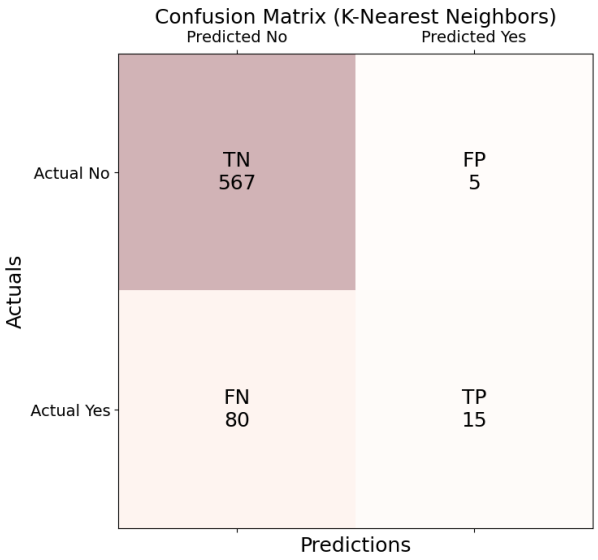


Figure 21: KNN confusion matrix (test set).

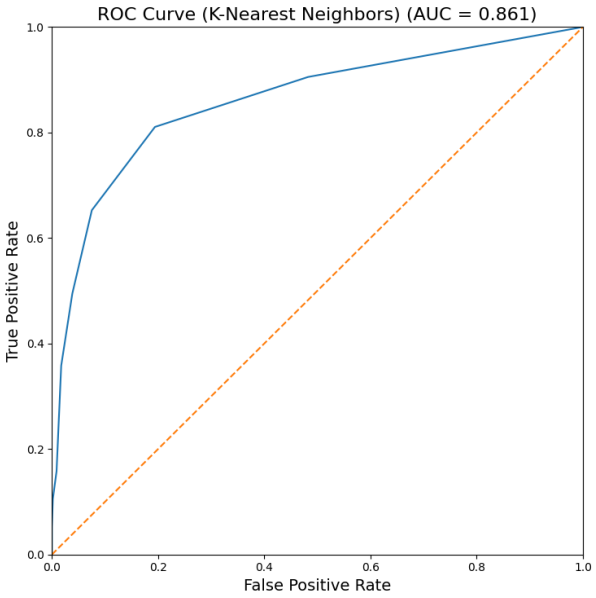


Figure 22: KNN ROC curve (test set).

4.5 Combined results

Table 2: Model performance summary (test ROC AUC), sorted by AUC (descending).

Model	Hyperparameters	Accuracy_train (%)	Accuracy_test (%)	AUC
RandomForestClassifier	`max_depth=10, n_estimators=50`	97.75	93.85	0.931
SVC	`C=1`	94.11	91.60	0.930
BaggingClassifier	`n_estimators=54`	99.96	95.80	0.920
KNeighborsClassifier	`n_neighbors=11`	88.71	87.26	0.861
DecisionTreeClassifier	`min_samples_split=10`	97.60	93.25	0.855
LogisticRegression	`max_iter=5000`	86.20	85.31	0.826
RidgeClassifier	`alpha=0.1`	86.12	85.61	0.819

Table 2 summarizes the predictive performance of all evaluated classifiers on the test set, ranked by ROC AUC. Overall, ensemble and margin-based methods perform best, while the linear baselines provide competitive but weaker discrimination.

The **Random Forest** model achieves the highest discrimination performance with an AUC of **0.931** and a test accuracy of **93.85%**. Very close behind, the **SVM** model also performs strongly (AUC = **0.930**, test accuracy = **91.60%**), indicating that a non-linear

decision boundary captures meaningful structure in the feature space. The **Bagging Classifier** ranks third (AUC = **0.920**) and attains the highest test accuracy overall at **95.80%**, suggesting that the default classification threshold leads to fewer misclassifications even though its ranking performance across thresholds is slightly below Random Forest and SVM.

Among the remaining models, **K-Nearest Neighbors** shows moderate discrimination (AUC = **0.861**) with test accuracy of **87.26%**. The single **Decision Tree** yields similar test accuracy (**93.25%**) but lower AUC (**0.855**), which is consistent with the higher variance and threshold sensitivity of a single-tree model relative to ensemble methods.

Finally, the linear models provide the weakest ROC AUC values: **Logistic Regression** achieves an AUC of **0.826** (test accuracy **85.31%**), while the **Ridge Classifier** performs slightly worse with AUC = **0.819** (test accuracy **85.61%**). These results imply that the relationship between the predictors and churn is not purely linear and that models capable of capturing interactions and non-linearities offer substantial gains in discriminative ability.

In summary, based on test-set AUC, the **Random Forest** provides the strongest overall performance, with **SVM** and **Bagging** as close alternatives. Given the class imbalance in churn prediction, AUC is the primary criterion for model comparison, and the top-performing models are therefore the preferred candidates for churn risk scoring.

5 Future work

Future work should (i) evaluate models with retention-oriented metrics under class imbalance (precision/recall/F1) and tune the decision threshold to match operational constraints; (ii) test robustness by replicating the workflow on other churn datasets/industries to assess generalizability and consistent feature drivers; and (iii) adopt deployment-focused practices such as cross-validation, probability calibration, and cost-sensitive learning to better reflect misclassification costs.

6 Conclusion

Because churn is the minority class, accuracy is not an informative metric on its own: several models achieve high accuracy while still missing many churners (false negatives), which is costly for retention. Therefore, models were compared mainly using ROC–AUC, which is less sensitive to class imbalance and better reflects how well churners are separated from non-churners across thresholds. By this criterion, non-linear and ensemble methods performed best: Random Forest and SVM achieved the highest AUC, and Bagging combined strong test accuracy with high AUC, indicating robust generalization. Overall, the results suggest churn is driven by non-linear relationships and interactions that are better captured by ensemble and margin-based models than by linear baselines.

References

[1] GeeksforGeeks. How to plot roc curve in python. GeeksforGeeks Article, July 2025. Last updated: 2025-07-23; Accessed: 2025-12-19.

[2] mnassrib. Customer churn prediction: Telecom churn dataset (model performance metrics section). Kaggle Notebook. Accessed: 2025-12-19.

[3] mnassrib. Telecom churn dataset. Kaggle Dataset. Accessed: 2025-12-19.

[4] The scikit-learn developers. Feature importances with a forest of trees. scikit-learn Documentation (Example Gallery).